LOGIN APPLICATION URL

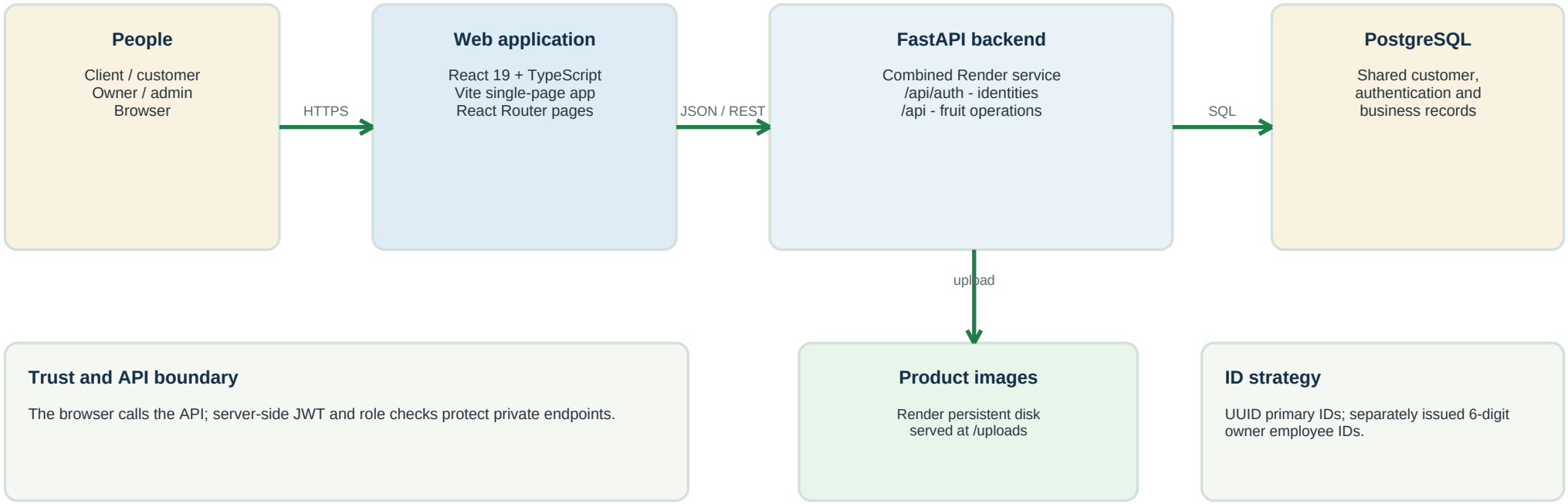
https://fruits-salling-app.vercel.app/login

INDKAN / FRUIT HOUSE

Complete Project Architecture

React and TypeScript frontend, Python FastAPI APIs, PostgreSQL persistence, client and owner authentication, and persistent product-image storage.

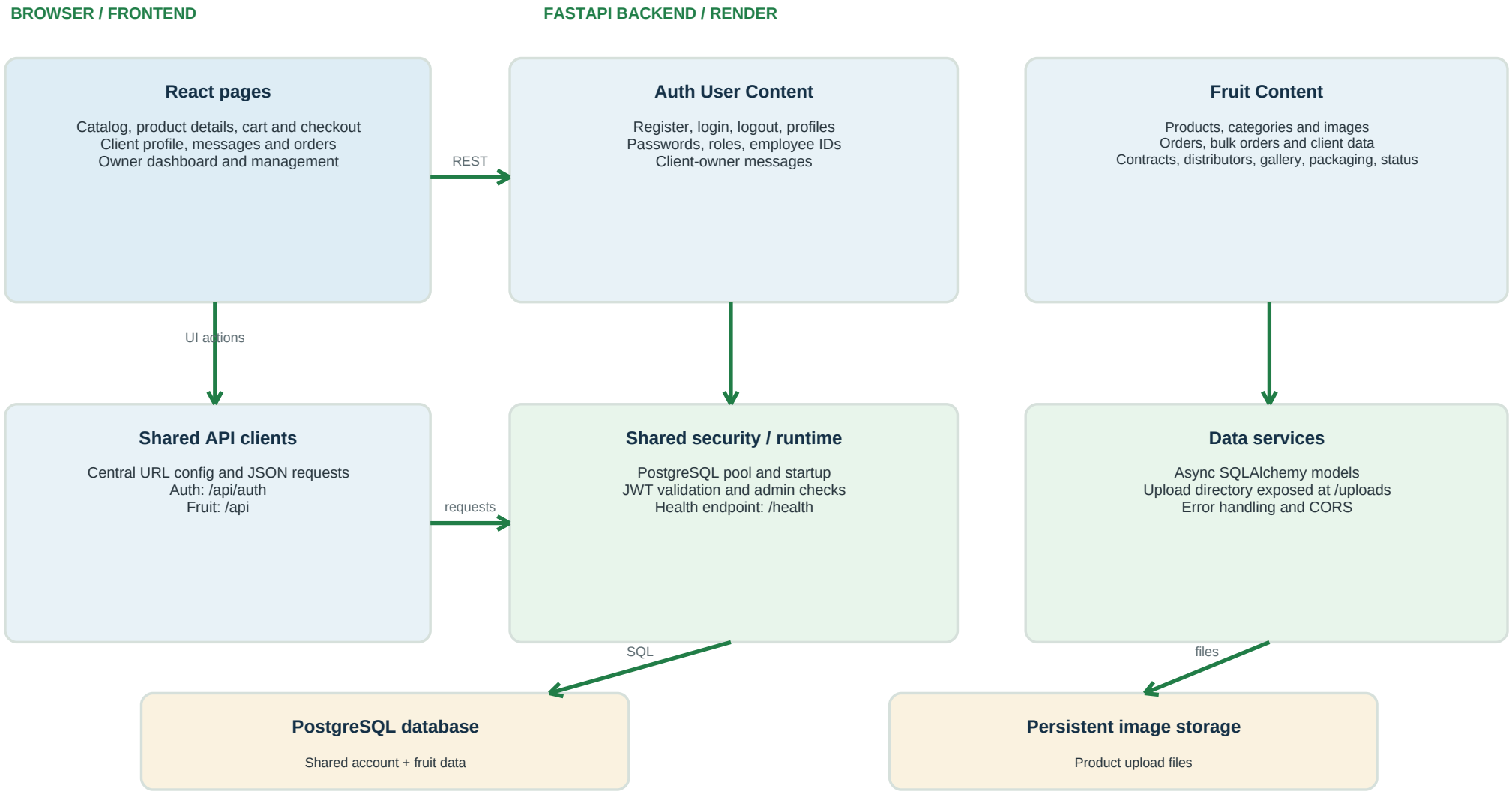
SYSTEM OVERVIEW



Architecture and behavior documented from the repository plus the deployment URLs supplied for this document.

Components, responsibilities & data flow

The frontend is a browser SPA. Both backend modules run separately in local Compose and are assembled into one FastAPI app for the Render API deployment.



Client and owner login / registration

Both account types use the same login page and /api/auth/login endpoint. The server recognizes the account type from the email and checks the corresponding credential record.

CLIENT ACCOUNT

Register with a personal Gmail address ending in @gmail.com and a password (8-128 characters). The server creates a customer-role account. Existing clients sign in with that Gmail address and password.

OWNER / ADMIN ACCOUNT

An admin first issues a one-time employee ID and matching @indkan.com email. Registration must use that exact, unused email and ID. Successful registration creates an admin-role account. Supported legacy @indikan.com addresses are recognized by login.

COMMON LOGIN AND AUTHORIZATION FLOW

1. **Enter credentials**
   Login form posts email + password to /api/auth/login.
2. **Verify identity**
   Backend selects the owner/client credential record; verifies salted PBKDF2-SHA256 password hash.
3. **Issue session**
   Returns a signed HS256 JWT (7-day expiry); PostgreSQL stores its hash, expiry and revocation state.
4. **Use protected APIs**
   Browser stores token in localStorage; sends Bearer token. API checks signature, session and role.

After login

Admin role routes to /owner; client role routes to /home. Logout calls the API to revoke the session, then clears browser token and role.

Security and data

Passwords are never stored as plain text. The server requires a JWT secret of at least 32 bytes. Expired, revoked or invalid sessions cannot access protected routes.

ID generation and account identity

Customer identity, employee-issued IDs, business-record IDs, contract numbers and JWT/session identifiers are distinct values with different purposes.

1. Customer account ID

On registration the backend creates customer_id using Python uuid4() (UUID v4). This UUID is the customer table primary key and is carried in the JWT subject. New business UUID primary keys (for example products, orders and order items) also use uuid4() defaults.

2. Contract number

If no contract number is provided, registration derives one from the account ID: CLI- + uppercase UUID hex for clients; OWN- + uppercase UUID hex for admins. This is a separate, human-facing account reference, not the database primary key.

3. Admin-issued owner employee ID

An authenticated admin submits employee details. The backend generates a cryptographically secure random six-digit value from 100000-999999, stores it as the employee ID primary key, and retries collisions (up to 100 attempts). It builds an email from normalized first name + joining year + normalized employee type + ID, e.g. first.2026.manager.123456@indkan.com.

4. One-time use + login

The employee ID is paired with its generated email in PostgreSQL. Owner registration must match that exact email; the ID is locked and marked used so it cannot be claimed twice. The account receives a UUID customer_id and OWN contract number. Login then uses the issued email and chosen password; employee ID is not a password.

Also generated: each JWT has a UUID jti; each auth session has its own UUID session_id; uploaded image filenames use random UUID hex.

Hosting, environment URLs & runtime

The listed public URLs are reproduced from the requested references. The client application uses the Vercel site; the API client defaults to the combined Render backend.

Login Application URL	https://fruits-salling-app.vercel.app/login
Primary Project URL	https://fruits-salling-app.vercel.app/
Target Reference / Web Address	https://fruits-salling-app.vercel.app/
Production Auth Portal	https://fruits-salling-o3cqyehzf-sudha19.vercel.app/
Main Vercel Deployment	https://fruits-salling-app-git-main-sudha19.vercel.app/
Render API Service	https://fruits-salling-application.onrender.com
Source Repository	https://github.com/sudhamayBala/e-commerce-website

PRODUCTION

Vercel serves the React/Vite SPA, including /login, /register, /home and /owner. The frontend calls Render at /api/auth and /api. Render starts one combined FastAPI backend, checks /health, connects to PostgreSQL via DATABASE_URL and stores uploads on a persistent disk.

LOCAL DEVELOPMENT

Docker Compose runs frontend :2000, auth API :3000, fruit API :4000 and pgAdmin :5050. Both APIs share PostgreSQL on :5432. Render variables include DATABASE_URL, JWT_SECRET and CORS_ORIGINS; secrets belong in private environment settings.

Some frontend utilities persist cart/order-history and selected dashboard state in browser localStorage; those local values are not all synchronized to PostgreSQL.